

```

// Import necessary objects and libraries
import { CSS3DObject } from '../libs/three.js-
r132/examples/jsm/renderers/CSS3DRenderer.js';
const THREE = window.MINDAR.IMAGE.THREE;

// Function to create a YouTube player
const createYoutube = () => {
  return new Promise((resolve, reject) => {
    var tag = document.createElement('script');
    tag.src = "https://www.youtube.com/iframe_api";
    var firstScriptTag = document.getElementsByTagName('script')[0];
    firstScriptTag.parentNode.insertBefore(tag, firstScriptTag);

    const onYouTubeIframeAPIReady = () => {
      const player = new YT.Player('player', {
        videoId: 'M7lc1UVf-VE',
        events: {
          onReady: () => {
            resolve(player);
          }
        }
      });
    };
    window.onYouTubeIframeAPIReady = onYouTubeIframeAPIReady;
  });
}

// Wait for the DOM content to be fully loaded before executing the code
document.addEventListener('DOMContentLoaded', () => {
  // Define an asynchronous function to start the AR experience
  const start = async () => {
    // Create a YouTube player using the createYoutube function
    const player = await createYoutube();

    // Create a MindARThree instance with the specified container and image target
    source
    const mindarThree = new window.MINDAR.IMAGE.MindARThree({
      container: document.body,
      imageTargetSrc: '../assets/targets/course-banner.mind',
    });

    // Extract the renderer, CSS renderer, scene, CSS scene, and camera from the
    MindARThree instance
    const { renderer, cssRenderer, scene, cssScene, camera } = mindarThree;

    // Create a CSS3DObject from an HTML element with the ID "ar-div"
    const obj = new CSS3DObject(document.querySelector("#ar-div"));
  };
});

```

```

// Add a CSS anchor to the MindARThree instance and attach the CSS3DObject to it
const cssAnchor = mindarThree.addCSSAnchor(0);
cssAnchor.group.add(obj);

// Define actions when the AR target is found and lost
cssAnchor.onTargetFound = () => {
  player.playVideo();
}
cssAnchor.onTargetLost = () => {
  player.pauseVideo();
}

// Start the MindARThree AR experience
await mindarThree.start();

// Set up a rendering loop using the CSS renderer to continuously render the CSS
scene
renderer.setAnimationLoop(() => {
  cssRenderer.render(cssScene, camera);
});
}

// Call the start function to begin the AR experience when the DOM content is fully
loaded
start();
});

```

Tutorial 6-YouTube

```
import { CSS3DObject } from
'../libs/three.jsr132/examples/jsm/renderers/CSS3DRenderer.js';
const THREE = window.MINDAR.IMAGE.THREE;
```

1. Description:

Import the CSS3DObject for rendering HTML elements in 3D and assign the THREE.js library from the MindAR framework.

```
const createYoutube = () => {
  return new Promise((resolve, reject) => { ... });
};
```

2. Description:

Defines a function to dynamically load the YouTube iframe API and create a YouTube player

.

```
document.addEventListener('DOMContentLoaded', () => { ... });
```

3. Description:

Ensures the script runs only after the HTML document is fully loaded.

```
const start = async () => { ... };
```

4. Description:

Defines an asynchronous function to initialize and start the AR experience.

```
const player = await createYoutube();
```

5. Description:

Waits for the YouTube player instance to be ready before proceeding.

```
const mindarThree = new window.MINDAR.IMAGE.MindARThree({ container:
document.body, imageTargetSrc: '../assets/targets/course-banner.mind' });
```

6. Description:

Initializes a MindARThree instance for AR, specifying the container and image target.

```
const { renderer, cssRenderer, scene, cssScene, camera } = mindarThree;
```

7. Description:

Extracts main components (renderer, scenes, and camera) from the MindARThree instance.

```
const obj = new CSS3DObject(document.querySelector('#ar-div'));
```

8. Description:

Converts an HTML element with ID 'ar-div' into a 3D object for AR rendering.

```
const cssAnchor = mindarThree.addCSSAnchor(0);
cssAnchor.group.add(obj);
```

9. Description:

Binds the CSS3DObject to an AR anchor and adds it to the anchor's group.

```
cssAnchor.onTargetFound = () => { player.playVideo(); };
cssAnchor.onTargetLost = () => { player.pauseVideo(); };
```

10. Description:

Defines actions to play or pause the video when the AR target is found or lost.

```
await mindarThree.start();
```

11. Description:

Activates the AR experience and starts processing.

```
renderer.setAnimationLoop(() => { cssRenderer.render(cssScene, camera); });
```

12.Description:

Sets up a rendering loop to continuously render the AR scene using the camera.

```
start();
```

13.Description:

Executes the AR setup and rendering process once the DOM is fully loaded.